

BillerBay Master PRD, Design System, Build Plan, EDI Plan, and Interactive Mock-up

Patients, Visits, Claims, Billing Work Queues, Payments, Settings, Audit, EDI, Code Sets, and Implementation Tasks

Prepared as a Claude / engineering handoff package

Artifact	Included
Master PRD DOCX	This document. Full product requirements, design rules, workflow states, backlog, EDI/code-set plan, acceptance criteria.
Markdown export	Plain-text version for repository, issue tracker, Claude prompts, and implementation planning.
Interactive HTML mock-up	Single-file clickable prototype: <code>BillerBay_Master_PRD_Interactive_Mockup.html</code> .
Design source of truth	<code>design.md</code> / BillerBay Command UI governs visual design, components, UX writing, states, accessibility, privacy, and security UX.

Version: 1.0 | Date: 2026-06-29

1. Executive Summary

BillerBay is a billing-team-first EHR / EMR / Practice Management MVP that replaces OfficeAlly-style fragmented workflows with a connected, high-visibility operating system for medical billing and practice operations. The MVP focuses on the revenue-cycle chain: patient identity and insurance, visit and charge capture, claim creation and validation, claim submission, payer response, payment posting, A/R follow-up, patient balances, settings, and audit history.

- The product must preserve the business objects users know from OfficeAlly: Patients, Visits, Claims, Payments, Applied Payments, ERA Auto-Post, Providers, Facilities, Payers, CPT/DX codes, and billing settings.
- The product must not copy OfficeAlly screen-by-screen. It must modernize the experience around connected context, queues, validation, lifecycle visibility, keyboard speed, and safe financial actions.
- The product must follow design.md exactly. BillerBay Command UI is not optional styling; it is part of the product requirements.
- The MVP is not full scheduling, full clinical charting, eRx, portal, labs, inventory, or advanced BI. Those are future phases unless specifically added by leadership.

Product principle	Meaning
Work queue first	The default user experience shows what needs attention, why, who owns it, and the next action.
One record, full context	Patient, visit, claim, payment, EDI event, and audit trail must be linked forward and backward.
Validation before submission	Dirty claims should be blocked or quarantined before they reach clearinghouse or payer.
Safe speed	Keyboard-first, bulk-capable, but no risky financial, claim, export, or settings action without confirmation and audit.
OfficeAlly familiarity without OfficeAlly friction	Use familiar objects and fields, but replace passive pages with modern BillerBay workflows.

2. MVP Scope

Included in MVP	Excluded from MVP
Patients, demographics, guarantor, insurance, eligibility summary, patient balance, related visits/claims/payments/activity	Full appointment scheduling and provider calendar automation
Visit list, visit detail, visit billing info, billing options, charge lines, diagnosis codes, claim linkage, balances	Full clinical charting templates and clinical decision support
Claim list, claim composer, CMS-1500 compatible data model, validation, submission, repair queue, secondary claims	ePrescribing, lab integration, imaging integration, patient portal
Billing command queues: ready visits, missing data, validation failures, rejected/denied claims, secondary, A/R, stale claims, patient balances	Inventory, marketing, full BI dashboards, MIPS/MACRA reporting
Payments/deposits, add payment, ERA auto-post, applied payments, unmatched/unapplied/reconciliation, reversals	Native credit card terminal integration beyond simple payment recording
Settings/master data: company, offices, providers, billing providers, referring providers, facilities, payers, CPT, DX, chart accounts, preferences, users, roles, security	Full document management beyond minimal billing attachments and proof records

3. End-to-End Revenue Cycle Chain

Patient contact / imported patient

- > Patient demographics and guarantor
- > Insurance and eligibility summary
- > Visit created
- > Visit info + billing info + billing options
- > Diagnoses + CPT/HCPCS charge lines
- > Claim created from visit
- > Claim validation and quarantine if blocked

- > 837P generation/submission
- > 999/997 and 277CA acknowledgment tracking
- > Payer adjudication
- > 835 ERA / EOB import
- > Payment, adjustment, denial, reversal, recoupment posting
- > Patient responsibility / secondary claim / A/R follow-up
- > Statement / collection / write-off / closed balance

4. Information Architecture

Navigation	Purpose	Primary flows
Dashboard	Operational landing page with queue counts and urgent work only.	Open high-risk queues, stale claims, unmatched ERA, ready-to-submit claims.
Patients	Identity, demographics, insurance, patient balance, related visits/claims/payments.	Search, add/edit patient, verify eligibility, add visit, view lifecycle.
Visits	Service/charge source before claims.	Add/edit visit, billing info, billing options, validate, create claim.
Claims	Submission and payer-response source.	Validate, repair, submit, batch submit, review lifecycle, print CMS-1500.
Billing	Team workbench; not a duplicate claims list.	Assign, fix, validate, submit, snooze, move to follow-up, bulk work.
Payments	Cash, ERA, posting, applications, reconciliation.	Import ERA, auto-post, fix exceptions, add manual payment, reverse/void.
Settings	Master-data and rules layer.	Providers, payers, facilities, CPT/DX, chart accounts, preferences, roles, audit.

5. BillerBay Command UI Requirements from design.md

All screens, components, copy, and behaviors must follow BillerBay Command UI from design.md. Developers and AI coding tools must not reinterpret the product as generic SaaS.

Area	Requirement
Brand colors	Deep ink #12122C for shell/anchor, electric blue #0410BD for primary actions, vivid blue #3F4CFF for hover/focus, white/soft lavender/cyan surfaces, lavender-gray borders.
Typography	Use Inter; tabular numbers for currency, dates, IDs, counts, and financial values.
Layout	Modern enterprise shell: top bar, workspace selector, universal search, command palette, notifications/help/user menu, left navigation, page header, main workspace, detail/record pages, activity timeline.
Tables	Every major list has search, filters, saved views, result count, pagination/virtualization, bulk actions, row actions, empty/loading/error states.
Forms	Labels, helper text, validation, error summary, save states, downstream impact copy for settings, no placeholder-only fields.
Modals/workflows	Guided workflows for claim submission, add payment, ERA auto-post, batch actions; include stepper, validation, review, submit/post, confirmation.
Drawers	Use drawers for fast inspection, exceptions, row detail, and cross-context review. Full patient/visit/claim editing uses record detail pages when the user needs complete context.
Status badges	Consistent statuses across visits, claims, payments, work items, settings, sync/jobs; never rely on color alone.
Security/privacy UX	Mask sensitive data, audit reveals/exports/payment edits/claim submissions/settings changes, require confirmation for high-risk actions.
Accessibility	Keyboard accessible, visible focus, WCAG 2.2 AA target, semantic HTML, ARIA where needed, no mouse-only workflow.

Core token	Value	Usage
bb.brand.ink	#12122C	Top bar, shell, headings, dark panels
bb.brand.blue	#0410BD	Primary buttons, selected navigation, primary actions
bb.brand.vividBlue	#3F4CFF	Hover/focus, selected outlines
bb.brand.teal	#007998	Secondary/info highlights
bb.surface.app	#F2F2F8	App background
bb.surface.white	FFFFFF	Cards, tables, modals, forms
bb.surface.border	#E3E3F1	Dividers, table lines
bb.brand.slate	#676687	Muted text and metadata

6. Status Systems and State Machines

Object	Statuses
Visit	Draft, Open, Missing patient data, Missing insurance, Missing diagnosis, Missing charges, Ready for billing, Claim draft created, Primary claim submitted, Secondary claim pending, Billed, Paid, Closed, Void.
Claim	Draft, Validation needed, Validation failed, Ready to submit, Awaiting batch, Submitted, Accepted, Rejected, Denied, Partially paid, Paid, Secondary pending, Closed, Voided.
Payment	Draft, Imported, Matched, Partially matched, Unmatched, Ready to post, Posted, Posted with warnings, Unapplied balance, Reconciled, Reversed, Void.
Work item	Open, Assigned, In progress, Waiting, Snoozed, Resolved, Escalated, Closed.
EDI transaction	Created, Queued, Sent, Acknowledged, Accepted, Rejected, Parsed, Failed, Reprocessed.

7. Module Requirements

7.1 Patients

- Patient list columns: patient ID, name, DOB, age, sex, account number, primary/secondary insurance, account type, status, patient responsibility balance, last visit date, open claims count, alerts, last updated, row actions.
- Filters: name/ID/account/DOB/subscriber ID, status, account type, insurance, balance, last visit, facility, provider, missing insurance, missing demographics, has open claims.
- Actions: add/edit patient, add visit, view visits, view claims, verify eligibility, print demographics, mark inactive, merge duplicate with permission.
- Patient profile is a full page with tabs: Overview, Demographics, Insurance, Visits, Claims, Payments, Notes & Activity. Clicking a patient row should open this full page, not only a sidebar.
- Demographics must include OfficeAlly-style patient data fields plus sensitive masking for SSN/confidential health flags and audit on reveal/change.
- Insurance tab must support eligibility summary, primary/secondary/third insurance, guarantor, subscriber fields, authorization, visits authorized/used/left, effective dates, signature/release fields, and warnings for missing/inactive insurance.

7.2 Visits

- Visit list columns: visit ID, type, date, patient, DOB, provider, facility/office, status, charges, balance, responsibility, primary claim, secondary claim, print statement flag, last updated, assigned user, row actions.
- Visit detail is a full page with sections/tabs: Patient Summary, Visit Info, Billing Info, Billing Options, Claim Linkage, Payments & Balances, Notes & Activity.
- Visit info fields: patient search, visit date/type/reason/chief complaint, allergies, status, print statement flag, optional vitals, provider, office, facility, provider notes, minimal medications if already captured, structured visit notes.

- Billing info grid: ICD type, diagnoses A-L, CPT/HCPCS, POS, modifiers A-D, ICD pointers, DOS from/to, charge, units, insurance/patient payments, adjustments, balance, line notes, anesthesia and NDC fields.
- Billing behavior: CPT search autofills defaults, diagnosis pointer validates, balances recalculate immediately, empty lines hidden, duplicate line item supported, multiple DOS lines supported, submitted/paid line deletion warns.
- Billing options include referring/supervising/ordering provider, facility, billing/rendering provider, condition fields, onset/accident/hospital dates, prior auth, Medicaid resubmission, CLIA, mammography, ambulance, attachment, hospice, contract fields with progressive disclosure.

7.3 Claims

- Claim list columns: claim number, created date, form type, claim type, DOS range, patient, visit ID, provider, facility, total charges, primary/secondary payer, insurance type, secondary flag, status, validation, submission, last submitted, last payer response, balance, owner, row actions.
- Claim composer sections: summary, patient/insured, payer, diagnosis, service lines, providers/facility, claim options, validation issues, submission history, payments/adjustments, notes/activity, CMS-1500 preview.
- Validation runs on creation, edit, before submit, batch submit, and field changes. Issues include blocking error, warning, informational suggestion with field link and fix action.
- Validation checks: subscriber ID, payer ID, patient DOB/sex, insured relationship, provider/facility/billing NPI, diagnosis pointer, no diagnosis, no billable line, POS, CPT, charge/units, invalid/future dates, coverage date, auth, secondary primary-payment detail, total mismatch, unsupported submission type.
- Repairable claims/rejections are first-class queues with source, message, date, age, owner, suggested fix, next action, original values, related field, resubmit action, activity history.
- Batch submission separates ready, warning, and blocked claims; excludes blocked claims; updates status and activity automatically; shows what happens next.

7.4 Billing Work Queues

- Billing is the operational command queue. It surfaces work across visits, claims, denials, payments, A/R, patient balances, stale claims, and ERA exceptions.
- Queue cards: visits ready for billing, visits missing billing data, claims validation failed, claims ready to submit, claims awaiting batch, rejected claims, denied claims, secondary claims pending, payments needing posting, ERA unmatched, unapplied payments, no-activity claims, patient balances requiring statement review.
- Queue table columns: work item type, priority, patient, visit ID, claim number, DOS, payer, provider, facility, amount, status, reason, assigned to, age, next action.
- Actions: assign to me, add note, fix issue, open detail drawer, mark reviewed, create claim, validate claim, submit claim, move to follow-up, snooze with reason, bulk assign/validate/submit.
- Automatic work item rules: missing insurance, charges but no claim, diagnosis but no charge, validation failure, claim rejection/denial, unmatched ERA, unapplied payment, secondary needed, no payer response after configured days, patient balance after insurance posting.

7.5 Payments and Posting

- Payments/deposits list columns: payment date, created date, payer type, payer, method, description, check/auth number, amount, applied, unapplied, matched, posting status, where applied, receipt, source, created by, row actions.
- Add payment guided workflow: source, payer/patient selection, payment details, claim/visit matching, apply, review, post.
- Applied payments list: applied date, payment date, DOS, patient ID/name, visit ID, claim number, applied type, amount, payment ID, account number, payer, posted by, delete/reverse with permission.
- ERA auto-post: check number, payer on claim/check, payment amount, applied/unapplied/unmatched, payment type, adjustment type, ERA status, match confidence, exception count, next action.
- Unmatched ERA detail must show patient from ERA, claim number, DOS, CPT, paid amount, adjustment reason, possible matching claims, confidence score, suggested match, manual match, skip with reason.

- Posting supports insurance payment, patient payment, adjustment, contractual adjustment, credit, reversal, refund marker, secondary transfer, patient responsibility transfer. Financial actions are audited.

7.6 Settings and Master Data

- Settings groups: Practice setup, Users and access, Providers, Facilities and locations, Payers and insurance, Billing codes, Payments/accounting, Claim defaults, Security, Audit logs.
- Settings screens must explain downstream impact and whether changes apply to future records only.
- Provider setup includes NPI, taxonomy, state license, DEA, default office/facility/billing provider, payer-specific PIN/group numbers; sensitive data masked.
- Payer setup includes payer ID, eligibility payer name, claim submission method, payer-specific billing rules, active/inactive behavior, and inactive selection override reason.
- CPT/HCPCS settings include code, description, default POS, modifiers, charge, active status, effective dates, notes, optional specialty/payer override, import/export, audit.
- Diagnosis settings include code type, diagnosis code, description, billable status, active/inactive, search by code/description.
- Chart accounts map insurance payments, patient payments, adjustments, credits, write-offs, refund markers.

7.7 Audit, Activity, and Migration

- Audit events required for demographic/insurance edits, sensitive reveals, visit edits/voids, diagnosis and charge edits, claim validation/submission/void, payment creation/posting/reversal, adjustment posting, ERA auto-post, settings changes, role changes, export actions.
- Each audit event stores timestamp, actor, actor role, action, entity type/id, before/after where relevant, IP/device where available, result, reason/note where required.
- Migration imports patients, insurance, visits, claims, payments, applied payments, providers, billing providers, referring providers, facilities, payers, CPT, DX, chart accounts, preferences.
- Imported records store source system, source ID, import batch ID, imported at/by, and data quality status. Users can filter imported records with missing required fields.

8. Smart Billing Rules and Pain-Point Requirements

Requirement	Build behavior
Admit/discharge code logic	For hospital/facility clients, required admit/discharge fields block submission when missing. If admit code already exists for a patient stay, warn or block duplicate admit billing and suggest follow-up codes.
Date logic	No future DOS on claims. Validate DOS ranges, admission/discharge/onset dates, coverage dates, authorization dates, and filing deadlines before clearinghouse submission.
Modifier and laterality logic	Detect RT/LT mismatch with diagnosis laterality. Warn/block if left-side diagnosis is paired with right-side modifier or vice versa.
Modifier 25/59 quarantine	When multiple CPTs on same DOS require modifiers, block/quarantine claim until biller reviews line-level edits. Integrate NCCI PTP logic.
Missing superbills/visits	Compare scheduled/seen/imported visit counts against superbills/created visits/claims; alert if patients seen but no billing record exists.
Bucket reminders	Claims/visits placed in QA or manual hold buckets generate reminders after configurable days.
Hospice modifiers	If patient is in hospice on DOS, auto-suggest or apply required modifiers based on rule configuration.
Timely filing limits	Payer-level TFL settings create alert buckets as claims approach filing deadlines.
ERA exceptions and reversals	Auto-post clean ERA lines, isolate exceptions, make reversals/recoupments/PLB easy to review and post.
Unlimited filters/saved views	No artificial 300-row-style limitations. Use server-side pagination/virtualization and saved views.
Code-set intelligence	ICD/CPT/HCPCS active-date validation, primary-allowed validation, NCCI PTP/MUE checks, medical necessity rules where licensed/available.

Requirement	Build behavior
Lifecycle view	Claim-level clickable timeline from visit creation to claim creation, edits, submission, acknowledgments, denials, payments, adjustments, and closure.
Proof records	Store eligibility response metadata/snapshot when used to support billing decisions.
CPT/DX hover visibility	Claims list should show or preview CPTs, modifiers, and DX codes without opening full claim page.
Deductible tracker	Dedicated bucket/report for deductible remaining, especially for calendar-year follow-up.
Unified claim identifier	Patient Control Number, EDI claim identifier, and BillerBay claim ID should use the same consistent value whenever operationally feasible.

9. EDI, CMS-1500, Code Sets, and Python Library Plan

BillerBay should own its normalized claim, remittance, payment, validation, and audit models. Use open-source packages for parsing/validation where they are strong, but do not rely on any single library as a complete production clearinghouse substitute.

Need	Recommended actual library/source	BillerBay wrapper/module
X12 parsing/validation and 997/999 generation	pyx12	edi/validators/pyx12_validator.py; edi/acks/x12_999_ack_service.py
835 ERA parsing	edi-835-parser; compare with openx12 and py835	edi/parsers/x12_835_parser.py; payments/era_import_service.py
837P/837I parsing/import	openx12; healthcare-io as reference	edi/parsers/x12_837_parser.py; edi/normalized/normalized_claim.py
837P outbound generation	Custom BillerBay builder validated by pyx12 and clearinghouse tests; evaluate x12-python/x12-edi-tools only as spikes	edi/builders/x12_837p_builder.py; x12_envelope_builder.py; companion_guide_validator.py
CMS-1500 rendering	Prototype with fillpdf; production with ReportLab coordinate renderer using NUCC 02/12 instructions	claims/cms1500/cms1500_field_map.py; cms1500_pdf_renderer.py
ICD-10-CM	Official CDC/CMS files; optional simple-icd-10-cm helper	codes/importers/import_icd10cm.py; icd10_lookup.py
HCPCS Level II	Official CMS HCPCS public-use files	codes/importers/import_hcpcs.py; hcpcs_lookup.py
CPT	AMA-licensed CPT source/API; do not scrape	codes/importers/import_cpt_licensed.py; cpt_lookup.py
NCCI PTP/MUE	Official CMS NCCI PTP and MUE files	codes/importers/import_ncci_ptp.py; import_ncci_mue.py; ncci validators
CARC/RARC	Official X12 CARC and RARC lists	codes/importers/import_carc.py; import_rarc.py; payments/denial_mapper.py

```

backend/
edi/
  builders/x12_837p_builder.py
  builders/x12_envelope_builder.py
  validators/pyx12_validator.py
  validators/companion_guide_validator.py
  parsers/x12_835_parser.py
  parsers/x12_837_parser.py
  parsers/x12_999_parser.py
  parsers/x12_277ca_parser.py
  acks/x12_999_ack_service.py
  normalized/normalized_claim.py
  normalized/normalized_remittance.py
claims/
  validation/claim_validation_engine.py

```



```

cms1500/cms1500_field_map.py
cms1500/cms1500_pdf_renderer.py
payments/
era_import_service.py
era_autopost_engine.py
payment_application_service.py
reversal_service.py
recoupment_service.py
deposit_reconciliation.py
codes/
importers/import_icd10cm.py
importers/import_hcpcs.py
importers/import_cpt_licensed.py
importers/import_ncci_ptp.py
importers/import_ncci_mue.py
importers/import_carc.py
importers/import_rarc.py
billing_rules/
ncci_ptp_validator.py
ncci_mue_validator.py
modifier_validator.py
laterality_validator.py
admit_discharge_validator.py
date_logic_validator.py
timely_filing_validator.py
hospice_modifier_validator.py
secondary_claim_validator.py

```

10. Data Model Overview

Domain	Core entities
Identity/access	Tenant, Practice, Office/Location, User, Role, Permission, Staff Profile
Master data	Provider, Billing Provider, Referring/Supervising/Ordering Provider, Facility, Insurance/Payer, CPT/HCPCS Code, Diagnosis Code, Chart Account, Preferences
Patient	Patient, Patient Insurance, Guarantor, Eligibility Check, Patient Note, Patient Alert
Visits	Visit, Visit Note, Diagnosis Assignment, Charge Line, Visit Billing Options
Claims	Claim, Claim Line, Claim Validation Issue, Claim Submission, Claim Status Event, EDI Transaction
Payments	Payment, Payment Application, Adjustment, ERA File, ERA Payment, ERA Service Line, Reversal, Recoupment, Deposit Reconciliation
Operations	Work Item, Queue, Saved View, Activity Event, Audit Event, Background Job

11. Backend Architecture and API Groups

- Frontend: React + TypeScript using BillerBay Command UI components/tokens.
- Backend: Django / Django REST Framework or equivalent enterprise backend.
- Database: PostgreSQL. Use JSONB only for raw/auxiliary parsed payloads, not as substitute for core relational claim/payment state.
- Jobs: Celery or equivalent background queue with Redis/broker.
- File storage: secure object storage for EDI files, PDFs, attachments, imports/exports, proof snapshots.

- Search: PostgreSQL full-text for MVP; upgrade to OpenSearch only if needed.
- Architecture: modular monolith separated by identity, master data, patients, visits, claims, payments, work queues, audit/activity, integrations.

/api/patients
 /api/patients/{id}/insurance
 /api/patients/{id}/visits
 /api/patients/{id}/claims
 /api/visits
 /api/visits/{id}/billing-lines
 /api/visits/{id}/billing-options
 /api/claims
 /api/claims/{id}/validate
 /api/claims/{id}/submit
 /api/claims/batch-submit
 /api/edi/transactions
 /api/era/import
 /api/era/{id}/auto-post
 /api/payments
 /api/payments/{id}/applications
 /api/work-items
 /api/settings/providers
 /api/settings/billing-providers
 /api/settings/facilities
 /api/settings/payers
 /api/settings/cpt-codes
 /api/settings/diagnosis-codes
 /api/settings/chart-accounts
 /api/audit-events
 /api/activity-events
 /api/search

12. Build Backlog and Task Export

Epic	Must build
E1 - Design system foundation	Tokens, AppShell, PageHeader, Button, StatusBadge, DataTable, FilterBar, SavedViews, Drawer, Modal, Stepper, ActivityFeed, AuditLogTable, CommandPalette, Toasts, Empty/Loading/Error/Success states.
E2 - Identity/security/audit	Tenancy, users, roles, permissions, MFA-ready UX, session timeout, audit events, sensitive masking/reveal logging, export logging.
E3 - Master data/settings	Practice, offices, providers, billing providers, referring providers, facilities, payers, CPT/HCPCS, DX, chart accounts, preferences, security settings, downstream impact copy.
E4 - Patients	Patient CRUD, demographics validation, duplicate detection, insurance/guarantor, eligibility summary, patient profile tabs, patient activity.
E5 - Visits	Visit CRUD, visit status, billing info grid, billing options, diagnosis/charge lines, balances, visit-to-claim readiness, notes/activity.
E6 - Claims	Claim generation from visit, claim composer, CMS-1500 model, validation engine, claim lifecycle, submission workflow, repair queue, secondary handling.
E7 - EDI and clearinghouse	837P builder/export, pyx12 validation, raw EDI storage, 999/997 parser, 277CA parser, submission/status events, payer companion-rule layer.
E8 - Payments/ERA	Payments/deposits, add payment workflow, 835 import, ERA parsing, auto-posting, exceptions, applied/unapplied, reversals, recoupments, reconciliation.

Epic	Must build
E9 - Billing queues	Queue cards, work item table, automatic work item rules, assignment, snooze, next action, bulk validate/submit, A/R aging, TFL alerts.
E10 - Reports/import/export	Operational exports, A/R reports, claim lifecycle reports, import migration, source ID tracking, data-quality status, PHI/PII export confirmation.
E11 - Interactive prototype parity	Keep HTML mock-up aligned with PRD flows. Every major module must have a populated table, detail page, action path, validation state, and next action.

13. Acceptance Criteria

1. User can search across patients, visits, claims, payments, providers, facilities, payers, CPT/DX, and settings from global search/command palette.
2. User can process high-volume work from queues without repeated module switching.
3. Patient row opens a full patient profile with demographics, insurance, visits, claims, payments, notes, and activity.
4. Visit row opens full visit detail with visit info, billing info, billing options, claim linkage, payments/balances, and activity.
5. User can create a claim from a visit and see exactly why the visit is or is not claim-ready.
6. Claim validation blocks critical errors and links directly to fields that need fixes.
7. User can batch submit ready claims while blocked claims are separated and preserved.
8. Claim lifecycle shows visit creation, claim creation, edits, validations, submissions, EDI acknowledgments, payer response, payments, denials, adjustments, and closure.
9. User can import or record ERA/EOB payments, auto-post clean lines, and review unmatched/unapplied exceptions.
10. Payment posting supports reversals/voids/refunds/recoupments with audit trail and permission control.
11. A/R aging and TFL reminders create actionable buckets with owner, age, reason, and next action.
12. Settings screens show downstream impact and audit changes.
13. All sensitive and financial actions are audited.
14. Every table supports filters, saved views, result count, row actions, empty/loading/error states, and no artificial row-selection ceiling.
15. All screens follow BillerBay Command UI from design.md.

14. Interactive HTML Mock-up Requirements

The included mock-up file is BillerBay_Master_PRD_Interactive_Mockup.html. It is a single-file prototype intended for product review, Claude UI guidance, and front-end implementation reference. It must be kept in sync with this PRD.

- Clickable primary navigation: Dashboard, Patients, Visits, Claims, Billing, Payments, Settings, Audit where applicable.
- Patient rows open full patient profile pages, not only drawers.
- Visit rows open full visit detail pages.
- Claim rows open full claim composer/detail pages.
- Drawers are used for quick exception inspection, not as the only patient experience.
- Mock data must be populated enough to avoid an empty feeling: queue counts, payer names, CPT/DX codes, balances, denial reasons, ERA exceptions, settings records, audit events.
- Mock-up must show BillerBay tokens, tables, filters, tabs, badges, modals, command palette, guided workflows, and safe confirmations from design.md.
- Prototype state may be simulated in JavaScript and reset on refresh, but every key PRD flow must have a clickable path.

15. Claude / AI Implementation Contract

You are building BillerBay software. Follow this Master PRD and design.md exactly.
Do not reinterpret this as generic SaaS. Do not use random colors or Tailwind defaults.
Use BillerBay Command UI tokens, Inter typography, 4px spacing, dense operational tables, clear badges, keyboard-first workflows, saved views, right drawers, full record detail pages, modals, guided workflows, and audit/security UX.

Build the MVP around Patients, Visits, Claims, Billing, Payments, Settings, Audit, EDI, and code-set validation. Do not build full scheduling, full clinical charting, eRx, portal, labs, inventory, or advanced BI unless explicitly requested.

Every feature must include:

- default, hover, active, focus, disabled, loading, error, success, and empty states
- permissions and audit behavior
- backend API contract
- data model changes
- acceptance tests
- realistic mock data
- source references for healthcare code sets and EDI requirements

16. Source and Standards References

- BillerBay EHR / EMR / Practice Management MVP PRD - uploaded source requirements for Patients, Visits, Claims, Billing, Payments, and Settings.
- design.md - BillerBay Command UI design system, accessibility, security/privacy UX, component states, and copy standards.
- ClinicTraq Original UX Critique - workflow/UX critique emphasizing operational guidance, readiness, next actions, drawers/panels, validation, and avoiding passive screens.
- Pain Points Responses v2 - smart billing logic, NCCI, TFL, A/R reminders, lifecycle view, ERA/reversal issues, filters, and code-set intelligence requirements.
- pyx12 - HIPAA X12 parser/validator/converter; evaluates 997/999 usage.
- edi-835-parser / openx12 - Python candidates for 835 and 837 parsing evaluation.
- CDC/CMS ICD-10-CM files, CMS HCPCS public-use files, CMS NCCI PTP/MUE files, X12 CARC/RARC lists, NUCC CMS-1500 02/12 instruction manual, AMA licensed CPT content.